

Théorie des langages

Olivier Ridoux

automates finis

Motivation

- RE décrivent bien certains langages...
...mais **pas du tout leur reconnaissance**

$(a+b)^* \bullet (c+d)^* \bullet (a+b)^+ \bullet (c+d)^* \bullet z$

« abba...z »

« abba...ccd...dz »

« abba...ccd...baa...z »

par où commencer ?

Plan

- Théorie des automates finis
Finite State Automata (FSA)
Machine ($\mathcal{M}$)
- Expressivité des automates finis
- Mise en œuvre des automates finis

Théorie des automates finis

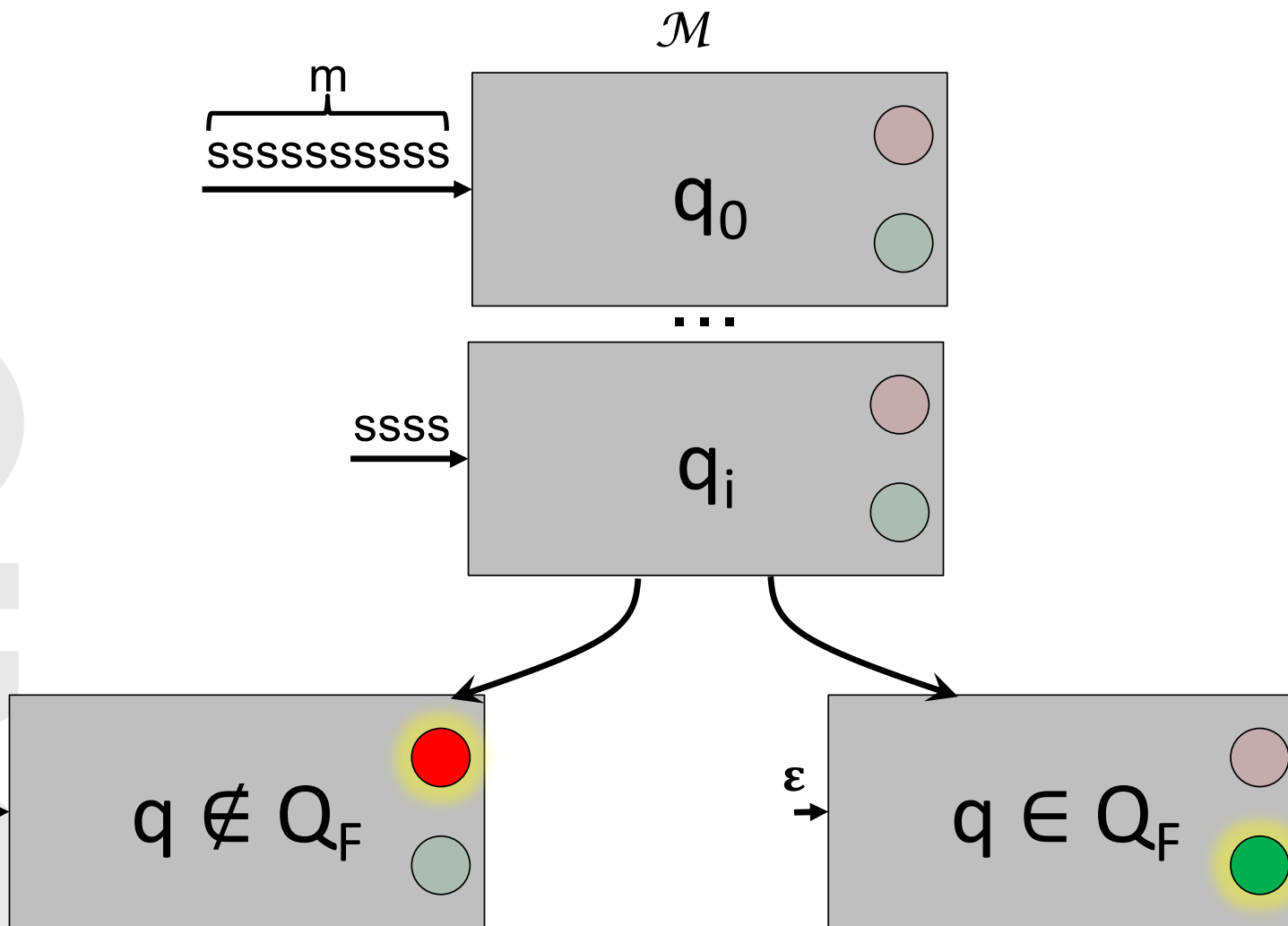
Automate $(\Sigma, Q, q_0, F, \delta)$ (1)

- Un **vocabulaire** fini : Σ
- Des **états** en nombre fini : Q
- Un état **initial** : $q_0 \in Q$
- Des états **finaux** : $F \subseteq Q$
- Des **transitions** : $\delta \subseteq Q \times \Sigma \times Q$
 $q \xrightarrow{a} q'$

À retenir absolument !

Automate (2)

- Un automate est une machine abstraite...
 - ...qui a un état qui prend sa valeur dans un vocabulaire fini
ouvert, fermé, libre, occupé, prêt...
 - ...qui lit une séquence de symboles d'un vocabulaire fini
 - ...qui la font changer d'état



Automate (3)

- Des **configurations** : ...
- Des **dérivations** : ...
- Des mots **acceptés** : ...
- Un langage **reconnu** : ...

À retenir absolument !

Configurations (1)

- Configuration $c \in Q \times \Sigma^* : (q, m)$
- Configuration **initiale** : (q_0, m)
- Configuration **finale** : $(q, \varepsilon) \mid q \in F$

Configurations (2)

- Une configuration combine...
...l'état (interne) de la machine
...et ce qui reste à lire (état externe, ou environnement)
- C'est le mot vide ou l'absence de transition définie
qui peut arrêter la machine...
...pas l'état final



= autorisation de s'arrêter
≠ obligation de s'arrêter

Dérivation (1)

- Dérivation immédiate

$$(q, a \bullet m) \rightarrow (q', m)$$

$$\text{ssi } q \xrightarrow{a} q' \in \delta$$

- Dérivation

$$(q, a_1 \dots a_n \bullet m) \rightarrow (q', m)$$

ssi

$$q \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \rightarrow \dots \xrightarrow{a_{n-1}} q_{n-1} \xrightarrow{a_n} q' \in \delta$$

Dérivation (2)

- Une dérivation synthétise une suite de changements d'état d'une machine...

...une histoire possible

Acceptation

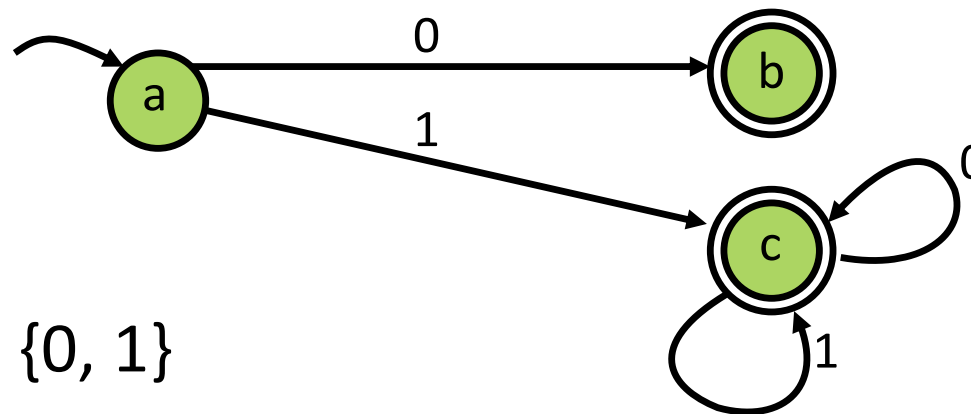
- Un automate $M = (\Sigma, Q, q_0, F, \delta)$
 - **accepte** un mot **m** ssi : **$(q_0, m) \rightarrow (q_f, \varepsilon)$** et **$q_f \in F$**
 - **reconnaît** un langage **$\mathcal{L}$**
ssi
tous les mots de **$\mathcal{L}$** et **seulement eux** sont **acceptés**

Automate (4)

- Un automate est une machine...
 - ...qui a lit des **mots**, une séquence de symboles d'un vocabulaire fini
 - ...qui les **accepte** si la lecture s'arrête dans un état final
 - ...qui **reconnaît** un **langage**, celui de tous les mots qu'elle accepte

Exemple - automate fini

$0 + 1 \bullet (0+1)^*$

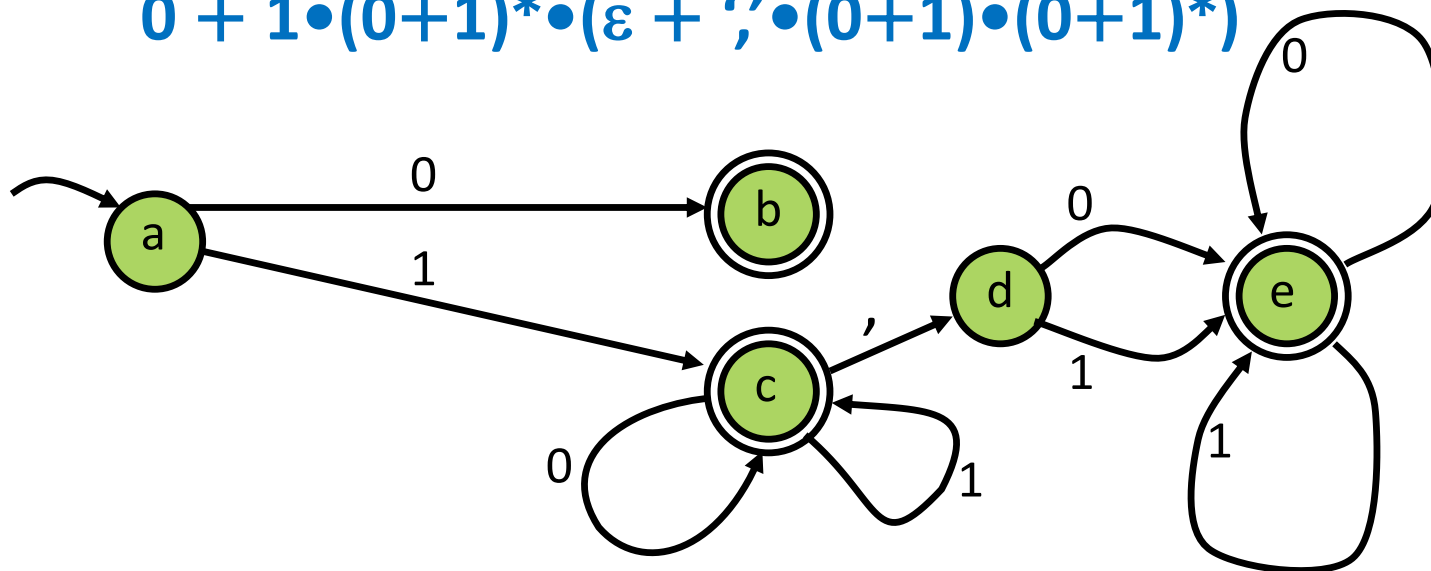


- $\Sigma = \{0, 1\}$
- $Q = \{a, b, c\}$ $q_0 = a$ $F = \{b, c\}$
- $\delta = \{(a,0,b), (a,1,c), (c,0,c), (c,1,c)\}$

Exemple - automate fini

- Notation binaire à virgule

$0 + 1 \cdot (0+1)^* \cdot (\epsilon + ' \cdot (0+1) \cdot (0+1)^*)$



Algorithme de décision de **M** accepte **m**

- Input : un mot **m** $\in \Sigma^*$
- Output : vrai ou faux

état $:= q_0$;

tant que **m** $\neq \varepsilon \wedge$ **état** $\in Q$ faire

soit $a \bullet m' = m$

état $:= \delta(\text{état}, a)$; **m** $:= m'$

fait

output **état** $\in F$



δ n'est pas toujours une fonction

- Si $\delta(q, a)$ n'est pas défini
 - **compléter** par
 $(q, a, \text{erreur}) \in \delta$ et $(\text{erreur}, v, \text{erreur}) \in \delta$
- Si $\delta(q, a)$ définit plusieurs q'
 - **déterminiser** M

from, letter $\rightarrow$ to DF ?

Automate (non) déterministe

- Automate **déterministe**

$$\text{ssi } \delta \in (Q \times \Sigma) \rightarrow Q$$

dépendance fonctionnelle **from, letter $\rightarrow$ to**

- Automate **indéterministe** sinon

Déterministe $\approx$ non déterministe

- Théorème : soit **M** non déterministe
alors
il existe **M_{det}** déterministe
tel que **M reconnaît $\mathcal{L}$** ssi **M_{det} reconnaît $\mathcal{L}$**
- Le non-déterminisme n'apporte rien
à l'expressivité des automates finis
penser à $\mathcal{P} = \mathcal{NP}$
- Un automate déterministe est parfois **beaucoup plus gros**
qu'un automate non-déterminisme équivalent

Principe de la preuve

- Soit $M = (\Sigma, Q, q_0, F \subseteq Q, \delta \in Q \times \Sigma \times Q)$
- Construire $M_{\text{det}} = (\Sigma, Q_{\text{det}}, q_{0\text{det}}, F_{\text{det}} \subseteq Q_{\text{det}}, \delta_{\text{det}} \in (Q_{\text{det}} \times \Sigma) \rightarrow Q_{\text{det}})$

$$Q_{\text{det}} = \mathcal{P}(Q)$$

$$q_{0\text{det}} = \{q_0\}$$

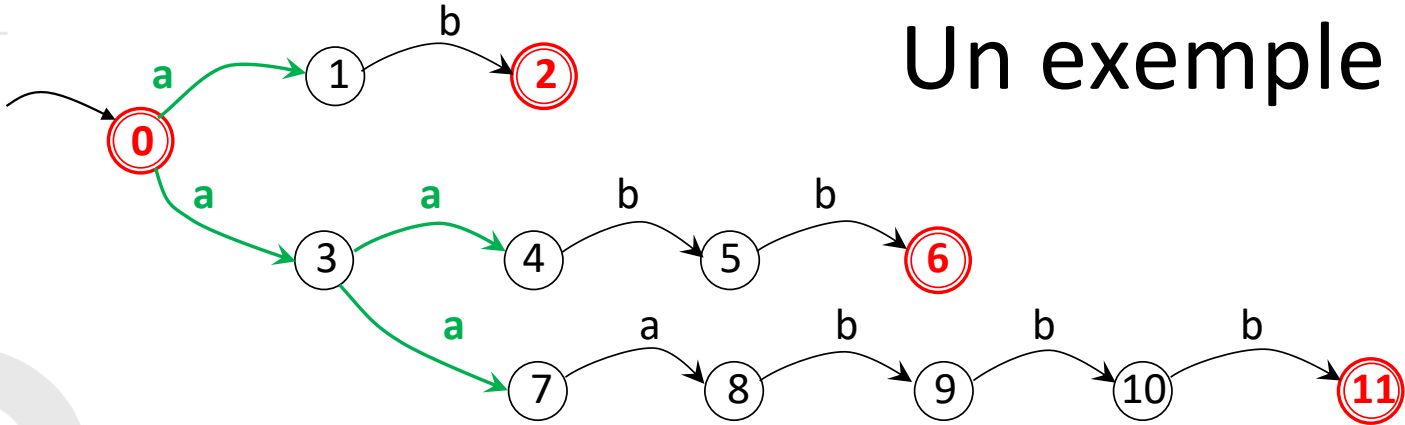
$$\delta_{\text{det}}(q_{\text{det}}, v) = \{q' \mid \exists q \in q_{\text{det}} \cdot (q, v, q') \in \delta\}$$

$$F_{\text{det}} = \{q_{\text{det}} \mid q_{\text{det}} \cap F \neq \emptyset\}$$

Algorithme

- Ne pas construire $Q_{\text{det}} = \mathcal{P}(Q)$ en totalité
coût exponentiel
- Construire Q_{det} à la demande
paresseusement
- Ne construire que la **partie de $\mathcal{P}(Q)$** qui est accessible depuis **$\{q_0\}$**

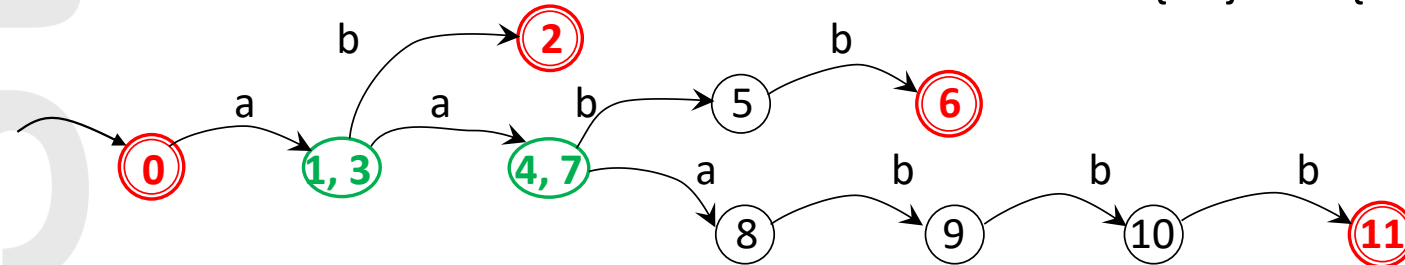
Un exemple


$$\{0\} \xrightarrow{a} \{1, 3\}$$
$$\{4, 7\} \xrightarrow{a} \{8\}$$

$\{5\} \rightarrow^b \{6\}$ final

$$\{1, 3\} \xrightarrow{a} \{4, 7\}$$
$$\{4, 7\} \rightarrow^b \{5\}$$
$$\{9\} \rightarrow^b \{10\}$$

$\{1, 3\} \rightarrow^b \{2\}$ final

$$\{8\} \rightarrow^b \{9\}$$
$$\{10\} \xrightarrow{b} \{\mathbf{11}\} \text{ final}$$


- Input : Σ , Q et δ
- Output : δ_{det} et la partie de $\mathcal{P}(Q)$ accessible depuis $\{q_0\}$

δ_{det} : $\emptyset$ au début, se remplit progressivement

Q_{det} : états de l'automate déterministe dont on connaît tous les voisins ;
 $\emptyset$ au début, se remplit progressivement

W_{det} : états de l'automate déterministe dont on ne connaît pas encore les voisins ; $\{q_0\}$ au début, puis fluctue en fonction de la découverte des voisins, est $\emptyset$ à la fin

tant que $W_{\text{det}} \neq \emptyset$ faire

soit $w_{\text{det}} \in W_{\text{det}}$

calculer toutes les transitions au départ de w_{det} ,

et tous les états accessibles par ces transitions,

rajouter les nouvelles transitions à δ_{det} et les nouveaux états à W_{det}

transférer w_{det} de W_{det} à Q_{det}

fait

- Input : Σ , Q et δ
- Output : δ_{det} et la partie de $\mathcal{P}(Q)$ accessible depuis $\{q_0\}$

$W_{\text{det}} := \{\{q_0\}\}; \delta_{\text{det}} := \emptyset; Q_{\text{det}} := \emptyset;$

tant que $W_{\text{det}} \neq \emptyset$ faire

soit $w_{\text{det}} \in W_{\text{det}}$

pour tout $s \in \Sigma$ faire

$\text{new}_{\text{det}} := \{q' \mid \exists q \in w_{\text{det}}. q \xrightarrow{s} q'\};$

$\delta_{\text{det}} = \delta_{\text{det}} \cup \{w_{\text{det}} \xrightarrow{s} \text{new}_{\text{det}}\};$

$W_{\text{det}} := W_{\text{det}} \cup \{\text{new}_{\text{det}}\};$

fait ;

$W_{\text{det}} := W_{\text{det}} \setminus \{w_{\text{det}}\};$

$Q_{\text{det}} := Q_{\text{det}} \cup \{w_{\text{det}}\};$

fait

Complexité

- **Exponentielle** dans le pire des cas
 - risque de devoir construire $\mathcal{P}(Q)$
 - rappel : $|| \mathcal{P}(Q) || = 2^{|| Q ||}$
- **Supportable** en pratique
 - algorithme très utilisé

Expressivité des automates finis

Théorème de Kleene

- $\exists M. M$ reconnaît $\mathcal{L}$
ssi
 $\mathcal{L}$ est rationnel
- M accepte m
= algorithme de décision de $m \in \mathcal{L}(re)$
- Preuve par construction de $re \leftrightarrow M$



Conséquence pratique

- Automates décrivent bien la reconnaissance d'un langage...
 - algorithme d'acceptation trivial

...mais pas bien le langage

...d'où

- RE pour la description
- traduction RE $\rightarrow$ [NDFSA $\rightarrow$] DFSA
- DFSA pour la reconnaissance

Variante

Automate à transition interne

- Comme avant
 - un vocabulaire : Σ
 - des états : Q
 - un état initial : $q_0 \in Q$
 - des états finaux : $F \subseteq Q$

...mais, transitions : $\delta \subseteq Q \times \Sigma \cup \{\epsilon\} \times Q$

$$q \xrightarrow{a} q' \text{ ou } q \xrightarrow{\epsilon} q'$$

transition « spontanée » sans lecture

Analogie

- Robinet thermostatique
- Distributeur automatique
 - consigne externe
 - ouvert, fermé, chaud, tiède, froid
 - insérer carte, saisir code, saisir montant, retirer carte
 - changement d'état interne
 - ouverture des arrivées d'eau chaude ou froide**
 - abandon sur *time-out***

Configurations

- Comme avant
 - configuration : $c \in Q \times \Sigma^*$, (q, m)
 - configuration initiale : (q_0, m)
 - configuration finale : $(q, \varepsilon) \mid q \in F$

Dérivation

- Comme avant

– dérivation immédiate :

$$(q, a \bullet m) \rightarrow (q', m) \text{ ssi } q \xrightarrow{a} q' \in \delta$$

- Plus **dérivation interne** :

$$(q, a \bullet m) \rightarrow (q', a \bullet m) \text{ ssi } q \xrightarrow{\varepsilon} q' \in \delta$$

- Dérivation :

$$(q, a_1 \dots a_n \bullet m) \rightarrow (q', m) \text{ ssi}$$

$$q \xrightarrow{i_1} q_1 \xrightarrow{i_2} q_2 \rightarrow \dots \rightarrow q_{m-1} \xrightarrow{i_m} q' \in \delta$$

$$\text{et } i_1 \bullet i_2 \bullet \dots \bullet i_m = a_1 \dots a_n$$

indéterminisme !

Acceptation

- Comme avant...

...un automate $M_{\text{int}} = (\Sigma, Q, q_0, F, \delta)$

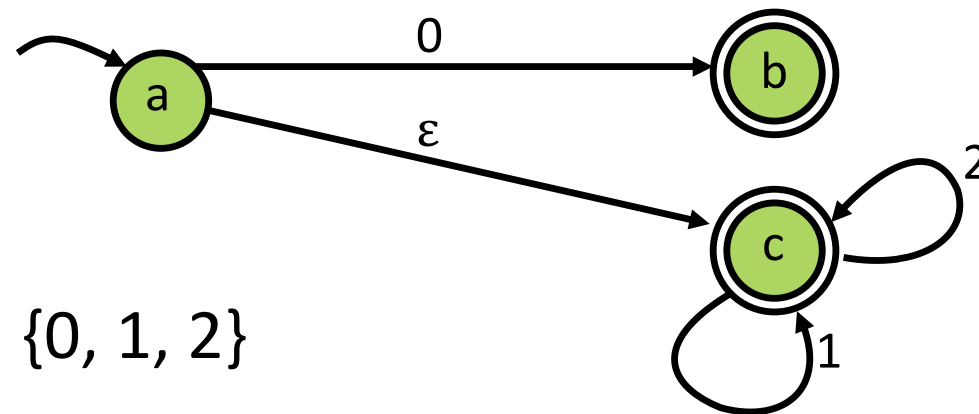
- **accepte** un mot **m** ssi :

$$(q_0, m) \rightarrow (q_f, \varepsilon) \text{ et } q_f \in F$$

- **reconnaît** un langage $\mathcal{L}$ ssi tous les mots de $\mathcal{L}$ et seulement eux sont **acceptés**

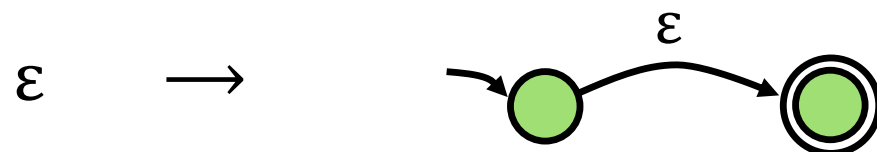
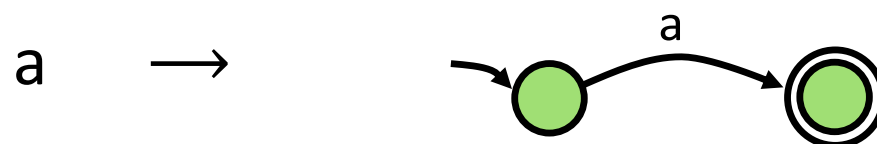
Exemple - automate fini

- $0 + (1+2)^*$



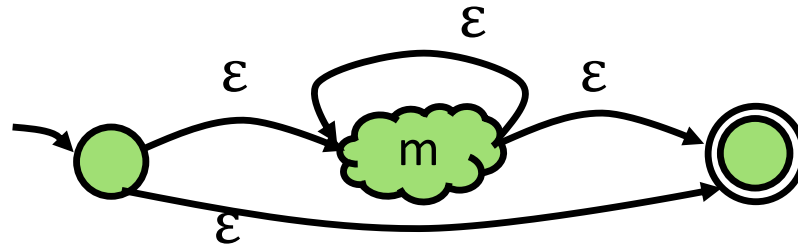
- $\Sigma = \{0, 1, 2\}$
- $Q = \{a, b, c\}$ $q_0 = a$ $F = \{b, c\}$
- $\delta = \{(a, 0, b), (a, \varepsilon, c), (c, 1, c), (c, 2, c)\}$

Application : $RE \rightarrow \mathcal{M} (1)$

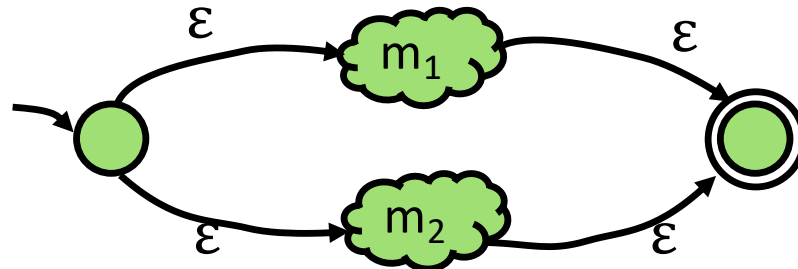


Application : $RE \rightarrow \mathcal{M} (2)$

$r^* \rightarrow$



$r_1 \mid r_2 \rightarrow$



$r_1 \bullet r_2 \rightarrow$



Complexité

- $RE \rightarrow \mathcal{M}$
 $re \mapsto m_\varepsilon \mapsto m$
- Construction de m_ε
complexité = $O(\|re\|)$
- Élimination des $\rightarrow^\varepsilon$
même algorithme que détermination
exponentiel dans le pire des cas...
complexité = $O(e^{\|m_\varepsilon\|})$

...supportable en pratique

Mise en œuvre des automates finis

Mise en œuvre (1)

- *Lexer* = **automate fini**
 - spécifié à l'aide d'expression régulières...
 - ...augmentées de déclaration des séparateurs...
 - ...ou décorées d'actions (**filtrer** ou retourner comme **token**)

Mise en œuvre (2)

- Table de transition augmentée

$$\delta(q,a) \rightarrow (q', \text{action})$$

Algorithme du lexer

- Input : un mot $m \in \Sigma^*$
- Output : vrai+tokens ou faux+erreur

état := q_0 ; accepte := vrai ; tokens := ε

tant que $m \neq \varepsilon \wedge$ accepte faire

soit pos, a, $m' = \text{lire}(m)$

état, action := $\delta(\text{état}, a)$; $m := m'$

accepte, tokens := action(pos,a)

fait

output accepte, tokens

callback

Conclusion (1)

- Expressions régulières **décrivent**
- Automates finis **reconnaissent**
 - notation d'algorithme

Conclusion (2)

- RE et automates finis parlent des même langages
 - Une méthodologie...
 - spécifier avec RE
 - transformer RE en $\mathcal{M}$
 - reconnaître avec $\mathcal{M}$
- ...cachée dans les outils

Conclusion (3)

- De nombreux algorithmes

accepte

$$\mathcal{M} \rightarrow \mathcal{M}_{\text{det}}$$

$$\mathcal{M} \rightarrow \mathcal{M}_{\text{ret}}$$

$$\mathcal{M} \rightarrow \mathcal{M}_{\text{min}}$$

$$\text{RE} \rightarrow \mathcal{M}$$

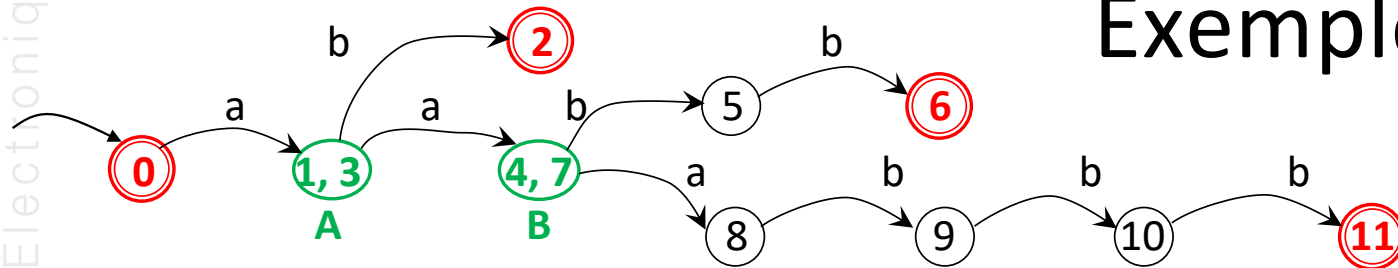
$$\mathcal{M} \rightarrow \text{RE}$$

...

$$M_{\text{min}} = (((M_{\text{ret}})_{\text{det}})_{\text{ret}})_{\text{det}}$$

En TD !

Exemple (1)



$$\{2, 6, 11\} \xrightarrow{b} \{A, 5, 10\}$$

$$\{A\} \xrightarrow{a} \{0\}$$

$$\{A, 5, 10\} \xrightarrow{a} \{0\}$$

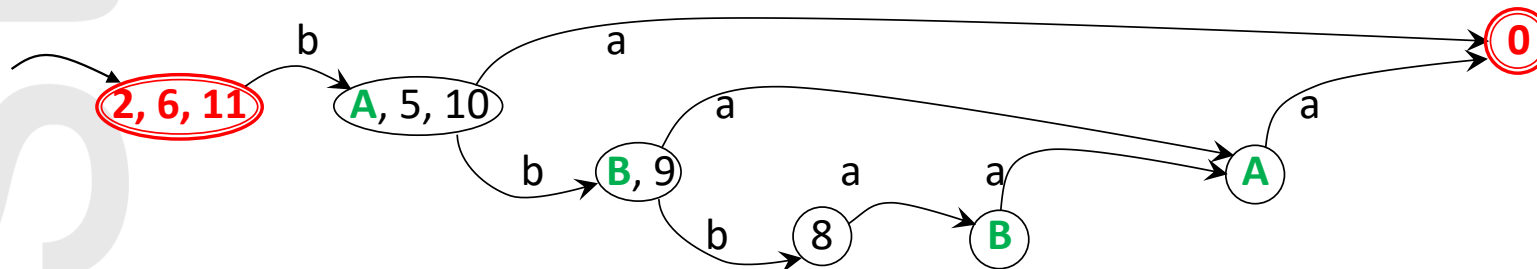
$$\{B, 9\} \xrightarrow{a} \{(1,3)\}$$

$$\{8\} \xrightarrow{a} \{B\}$$

$$\{A, 5, 10\} \xrightarrow{b} \{B, 9\}$$

$$\{B, 9\} \xrightarrow{b} \{8\}$$

$$\{B\} \xrightarrow{a} \{(1,3)\}$$



Exemple (2)

